

MIRA × Walker Reynolds "Digital Transformation 2026" — Alignment Analysis

Authored: 2026-06-01 **Author:** Claude (CHARLIE), commissioned by Mike Harper **Scope:** Map MIRA's actual codebase against Walker Reynolds' DT-2026 framework (11-point distillation supplied with the task). Assess what is built, designed, missing, and what the garage bench proves. **Method:** CodeGraph + grep + file reads across `mira-bots/`, `mira-hub/`, `mira-mcp/`, `mira-relay/`, `mira-bridge/`, `mira-crawler/`, `mira-pipeline/`, `plc/`, `mira-fault-detective/`, `docs/`, `mira-web/`. Six parallel evidence-gathering passes.

Status legend (from the master plan §1): ✔ built & working · ⚠ partial · designed, not built · bench-only (not customer-shipped) · simulated/mock (looks real, isn't)

0. The headline finding

MIRA has built Walker's *architecture* but is assembled back-to-front relative to Walker's *journey*.

Walker's framework is, at root, two things: an **architecture** (UNS as the 2D semantic hierarchy + Knowledge Graph as the 3D relationship layer, with AI agents plugged in on top) and a **journey** (always start with current state → connect → collect → store → analyze → visualize → find patterns → predict → sell your data).

- On the **architecture** axis, MIRA is genuinely strong and the alignment is *deliberate, not coincidental*. `docs/adr/0012` is literally titled "**MES Architecture — Walker Reynolds UNS Framework**" and is Accepted. `STRATEGY.md` codifies a "Walker rule" (infrastructure-first, AI second). The UNS resolver, the ltree path space, the `kg_relationships` table, and the `kg_impact_analysis / kg_root_cause_chain / kg_traverse_chain` MCP tools are a direct implementation of "3D knowledge graph relationships that let AI agents do root-cause across unrelated data points."
- On the **journey** axis, MIRA is inverted. Walker's mandatory **step 1 — live current state, "which assets are running right now?"** — is the *most simulated and stubbed part of the entire stack*. The brain (analyze/reason/ground) is built; the nervous system that feeds it live plant state (connect → collect → current-state) is bench-only or mocked.

The encouraging part: **MIRA's own master plan already agrees with Walker**. The dependency-true critical path (`docs/plans/2026-06-01-mira-master-architecture-plan.md` §D10) is: Phase 1 (schema) → **Phase 4 (tag collector)** → **Phase 5 (tag event stream)** → everything else. The plan starts with current-state capture. It just hasn't been built yet — the master plan's own header reads Status: DRAFT — planning only, no production code.

So this document's job is to hold four honesty lines while crediting real strength:

- Planned ≠ built.** Migrations `032_decision_traces` through `037_*` do not exist. A 14-phase roadmap is not a capability.
- Simulated ≠ real.** Fault Detective's flaky-wire detection — the most impressive-sounding capability — runs on REST-injected synthetic sensor data. `live_signal_events.simulated` defaults `TRUE`.
- Schema-exists ≠ data-flows.** `tag_entities` exists with no writer. Sparkplug B is a column, not a parser.
- Diagnostics ≠ prediction.** MIRA *grounds* answers; it does not *predict*. Walker's "find patterns → predict the future" is genuinely absent.

1. Point-by-point: Walker's 11 framework components vs. MIRA

Walker #1 — DT = making DATA the primary commodity (not just automating)

Alignment: STRONG in doctrine, PARTIAL in product.

`NORTH_STAR.md:3` states it almost verbatim: "FactoryLM is a maintenance digital transformation firm. We turn messy maintenance reality into AI-ready infrastructure. MIRA is the execution layer that runs on top." The asset being sold is the **structured namespace + knowledge graph** — i.e., the data — not a chat license. The "Knowledge Cooperative" flywheel (`NORTH_STAR.md`) explicitly treats accumulated OEM-manual parses, fault-pattern libraries, and PLC tag-mapping heuristics as the compounding commodity.

What's missing for "data as *the* commodity": MIRA today monetizes *answers* (\$97–\$499/mo for grounded troubleshooting), not *data products*. The data-as-sellable-asset step (Walker #3c) is doctrine, not plumbing — see Walker #3 below.

Walker #2 — Industry 3 → Industry 4 (automate processes → automate business DECISIONS)

Alignment: PARTIAL. MIRA automates a *decision support* loop, not yet *autonomous decisions*.

-  Industry-3 layer present on the bench: `plc/Micro820_v4.1.9_Program.st` is real ladder logic automating the conveyor process (5-state FSM, e-stop supervision, GS10 Modbus RTU command/poll).
-  Industry-4 layer (decisions from data): the diagnostic engine (`mira-bots/shared/engine.py` , class `Supervisor`) reasons over retrieved evidence and proposes fixes — but every consequential step is **human-gated by design** (UNS confirmation gate; `proposed` → `verified` KG transitions are a human action per TOO Invariant; no PLC writes in any customer-shipped path per ADR-0021). That's correct and safe, but it means MIRA *informs* business decisions rather than *automating* them. The `kg_flag_pm_mismatches` MCP tool (compares actual MTBF from work-order history to planned PM cadence) is the closest thing to an automated business-decision signal in the production stack.

Walker #3 — Three steps after going digital: (a) data-driven decisions, (b) digital supply chain, (c) sell your data

Sub-step	MIRA status	Evidence
(a) data-driven decisions	 partial	Engine grounds diagnoses in retrieved evidence; <code>kg_flag_pm_mismatches</code> flags plan-vs-actual. But decisions are advisory, and the live-data half is mocked (see #5).
(b) digital supply chain	 absent	No supplier integration, no parts-availability feed, no procurement loop. <code>mira-mcp</code> has CMMS work-order tools (<code>cmms_create_work_order</code> , <code>cmms_list_pm_schedules</code>) but nothing that reaches outside the plant to suppliers. This is a genuine blind spot.
(c) sell your data	 doctrine only	The "Knowledge Cooperative" (anonymized cross-plant PM/fault sharing) in <code>NORTH_STAR.md</code> is the sell-your-data concept. No mechanism exists: no anonymization pipeline, no data-product packaging, no marketplace. It's a flywheel slide, not code.

Walker #4 — Only 1 in 12 companies make the jump (huge market for enablers)

Alignment: STRONG in positioning, this is MIRA's explicit wedge.

`STRATEGY.md:83-89` positions MIRA precisely as the enabler for the 11 who fail: CMMS vendors (MaintainX/UpKeep/Limble) "*sell software to people who already have structured data. We're the firm that creates the data.*" UNS consultancies ("*Walker, 4IR, etc.*", named explicitly at `STRATEGY.md:85`) "*focus on production data (OEE, throughput). Maintenance is an afterthought.*" The differentiation claim: pure consultancies leave a binder; pure SaaS leaves you hallucinating; MIRA does the hands-on structuring **and** ships the execution layer. See §5 for whether the product actually delivers 12-of-12.

Walker #5 — Journey ALWAYS starts with CURRENT STATE ("which assets are running right now?"), first 12 weeks

Alignment: WEAK in the shipped product — this is the central gap.

This is Walker's load-bearing claim and MIRA's softest spot.

-  The capability *exists on the bench*: `plc/live-plc-bridge/bridge.py` ( BENCH-ONLY) polls the real Micro820 at `192.168.1.100:502` every 500 ms and publishes live GS10 frequency/current/voltage/DC-bus + motor/safety state to MQTT. `mira-relay/relay_server.py` ( , in `docker-compose.saas.yml`) receives HMAC-signed tag batches from Ignition into a SQLite `equipment_status` table, surfaced to the agent via `get_equipment_status` (`mira-mcp/server.py:160`).
-  But the Hub's current-state layer is **simulated by default**. Migration `019` (`live_signal_events`) ships with `simulated BOOLEAN DEFAULT TRUE` and a comment: "*For the May 21 demo we don't subscribe to MQTT — the simulator endpoint (POST /api/demo/signals/toggle) writes rows here ... so we never confuse fake samples for real telemetry once the real MQTT bridge lands.*" The real MQTT → NeonDB `live_signal_events` write path **does not exist**.
-  The first-class tag table `tag_entities` (Hub migration `025` , with `sparkplug_topic` / `opcua_node_id` / `uns_path` columns) **has no writer**.
-  The Command Center "live" dot is HTTP *display reachability*, not PLC signal freshness (`mira-hub/src/app/api/command-center/tree/route.ts` — server-side `probe()` of each display URL, 2 s timeout). Honest in the code, but worth stating: green ≠ "the asset is running," green = "the HMI page responded."

Net: MIRA cannot today walk into a plant and answer "which assets are running right now?" across the fleet from live data. It can on the *bench*, via bench-only tooling, for one cell. The master plan's Phase 4/5 close exactly this gap — and the plan correctly sequences them first (§D10).

Walker #6 — The stack: Cloud → ERP → MES → SCADA → PLC/HMI

Alignment: PARTIAL — MIRA covers Cloud↔SCADA↔PLC/HMI; ERP/MES out of scope by decision.

Stack layer	MIRA presence
Cloud	✅ <code>mira-pipeline</code> (:9099), <code>mira-hub</code> (<code>app.factorylm.com</code>), <code>mira-relay</code> cloud endpoint, NeonDB
ERP	❌ none — not in scope
MES	❌ deferred — ADR-0012 adopts the UNS framework but MES modules (OEE/throughput) are explicitly <i>not</i> the wedge; STRATEGY.md cedes production data to the UNS consultancies
SCADA	⚠️ via Ignition only — <code>ignition/webdev/FactoryLM/</code> (WebDev module, gateway tag-stream scripts), <code>mira-pipeline/ignition_chat.py</code> . Customer-shipped reads go <i>through Ignition</i> , never a direct MIRA→PLC socket (ADR-0021)
PLC/HMI	✅ bench (Micro820 + GS10 + Ignition Perspective <code>ConvSimpleLive</code>), 🟦 bench-only bridges

MIRA deliberately owns the **maintenance slice** of the stack (SCADA-down, plus the cloud reasoning layer) and cedes ERP/MES. That's a defensible focus, but against Walker's full-stack picture it's a partial footprint — worth naming so nobody claims more.

Walker #7 — Process: Connect → Collect → Store → Data Ops → Analyze → Visualize → Find Patterns → Report → Solve

This is the spine. Scored stage by stage:

Stage	Status	Where it lives / why
Connect	⚠️ / 🟦	<code>mira-relay</code> ✅ HMAC ingest, in <code>saas.yml</code>) + Ignition WebDev tag-stream ✅ gateway scripts at <code>ignition/webdev/FactoryLM/gateway-scripts/tag-stream.py</code>). Customer path is Ignition-only. Direct PLC connect is 🟦 bench (<code>live-plc-bridge</code>) or ❌ deferred (<code>mira-connect/</code> Modbus driver — scaffolded + tested, "Config 4", not deployed). MQTT broker (Mosquitto) is real but bench/dev. Sparkplug B is ❌ a schema column, not a parser — no <code>spBv1.0/...</code> decode anywhere.
Collect	⚠️ / 🟥	<code>equipment_status</code> SQLite ✅ (real, fed by Ignition/relay). Per-turn Ignition tag snapshot ✅, <code>_format_tag_preamble</code> in <code>ignition_chat.py</code>). Hub <code>live_signal_events</code> (🟥 simulator-fed). No fleet-wide live collection.
Store	✅	<code>knowledge_entries</code> (pgvector, 25k+ rows), <code>kg_entities</code> / <code>kg_relationships</code> , <code>ai_suggestions</code> , <code>tag_entities</code> (schema only), <code>display_endpoints</code> , <code>ignition_audit_log</code> . Two migration lineages reconciled by ADR-0013 (Hub canonical for product surface; <code>docs/migrations/</code> for engine KG). SQLite WAL for bridge/relay state. One real defect: uploads via Hub/MiraDrop write <i>only</i> Open WebUI KB, never <code>knowledge_entries</code> , so uploaded manuals aren't citable on any chat channel (<code>project_upload_retrieval_gap.md</code> , ADR-0020 fix pending).
Data Ops	⚠️	Chunk → embed → dedup → store pipeline (<code>mira-crawler/ingest/</code>) is ✅ but invoked only from bot chat; the first-class ingest API is ❌ (master plan Phase 2). MiraDrop watcher ✅ but on the OW path that's sunsetting.
Analyze	✅	<code>Supervisor</code> engine, 8-state FSM (<code>fsm.py</code>), 1–5 self-critique groundedness gate, <code>inference/router.py</code> cascade (Grok→Cerebras→Gemini, PII-sanitized default-on), BM25+pgvector RAG (<code>rag_worker.py</code> , <code>neon_recall.py</code> ; embedding gate fixed PR #1385). PLC live-tag worker is a stub (<code>plc_worker.py</code> is a no-op — deferred to Config 4).
Visualize	✅ / ⚠️	Hub Command Center (UNS tree + live HMI iframe), <code>/namespace</code> , <code>/proposals</code> , <code>/discovery</code> , <code>/assets/[id]/signals</code> are ✅ . Node-RED conveyor HMI ✅ but bench/demo. Ignition Perspective <code>ConvSimpleLive</code> ✅ on the PLC-laptop gateway.
Find Patterns	⚠️ / 🟥	Recurring-fault detection ✅ basic (<code>detection/recurring_fault.py</code> — LIKE-match, 2-in-30-days, fires a push). Fault Detective rule engine ✅ logic / 🟥 inputs (8 rules incl. flaky-wire, but sensor faults are REST-injected by <code>mira-fault-sim/sim.py</code>). <code>kg_flag_pm_mismatches</code> ✅ (MTBF vs PM). No SPC, no ML anomaly detection, no historian trending.
Report	⚠️ / 🟥	AI narrative endpoint <code>/api/reports/generate</code> ✅ (calls cascade) but on static seed WO data , not live NeonDB. Reports KPI dashboard 🟥 hardcoded mock values (<code>reports/page.tsx</code> : MTTR 2.4h, MTBF 312h — const arrays, Labs-flag-gated). Morning pass-down email ✅ (<code>morning_report.py</code> , Celery beat, Resend+ntfy).
Solve	⚠️	Grounded fix steps ✅ ; CMMS write-back ✅ (<code>cmms_create_work_order</code> across Atlas/MaintainX/Limble/Fiix). No closed-loop control (correct — out of scope per ADR-0021).

Reading the row: the right half of Walker's pipeline (Store→Analyze→Visualize) is MIRA's strength. The left half (Connect→Collect) and the predictive tail (Find Patterns→Report) are where the simulated/stubbed reality lives.

Walker #8 — UNS = the architecture; 2D semantic hierarchy (parents/children/siblings)

Alignment: **STRONG**.

- ✓ Path builders: `mira-crawler/ingest/uns.py` (`manufacturer_path` , `model_path` , `fault_code_path` , `manual_path` , `pm_schedule_path` , `slug()` , `RESERVED_LABELS`). ISA-95 ltree, lowercase, enforced by `.claude/rules/uns-compliance.md` .
- ✓ Message resolution: `mira-bots/shared/uns_resolver.py` (~900 lines, `UNSContext` dataclass with `source` + `confidence` band). One extraction point per turn; fault codes stripped before model candidates (the historical `powerflex 525 f0004` bug).
- ✓ Storage: `kg_entities.uns_path` (ltree), `tag_entities.uns_path` , `cmms_equipment.uns_path` .
- ✓ The 2D hierarchy (parent/child/sibling) is real: ADR-0018 places component instances as **tree siblings** under an asset, with control relationships (power, signal) living as KG edges, not a second hierarchy.

Caveat: the *live bench* uses a flat demo namespace (`demo/cell1/conveyor/cv101/...` in the MQTT topics) rather than a proper `enterprise.garage.demo_cell` ISA-95 path. The Hub tree shows real `enterprise.*` entities, but the live MQTT stream doesn't reconcile to them yet. So the UNS *schema* is fully Walker-compliant; the *live demo data* isn't wired into it.

Walker #9 — 2026 addition: Knowledge Graphs = 3D relationships (root-cause across unrelated data)

Alignment: **STRONG** in design, the single most Walker-aligned thing MIRA has built.

The MCP tool surface is almost a literal implementation of Walker's "3D relationships let AI agents do root-cause analysis across unrelated data points":

- `kg_root_cause_chain(tenant_id, fault_entity_id)` — walks `caused_by` edges, returns cause chain + sibling alternates from prior conversations.
- `kg_impact_analysis(tenant_id, entity_id)` — walks `feeds` forward, returns downstream nodes + blocked/partially-impacted lines.
- `kg_traverse_chain(tenant_id, start_entity_id, relationship_chain, max_depth)` — follows an arbitrary relationship-type sequence.
- `kg_maintenance_context(...)` — aggregates hierarchy + components + faults + WOs + parts + manuals + PM + plan-vs-actual mismatch into one bundle.

These are real, registered `@mcp.tool` functions in `mira-mcp/server.py` .

The honest gap (schema-exists ≠ data-flows): the graph is **richly queryable but thinly populated by real evidence**. Per the master plan Phase 3, ingest currently writes `kg_relationships` *directly* — the namespace-builder spec calls this "*rhetorical, not real*" because the `proposed → evidence → review → verify` loop isn't closed (`relationship_proposals` + `relationship_evidence` tables exist, Hub migration 018, but `kg_writer.upsert_relationship()` bypasses them). And nothing in the *running* stack auto-proposes a KG edge from a live fault event — KG enrichment is manual or photo-triggered, not runtime-fault-triggered. The 3D graph can answer root-cause questions; it just doesn't yet *grow itself* from the current-state stream the way Walker's vision implies.

Walker #10 — Post-agentic AI: trained models + agents plugged into digital infrastructure, inferencing on current state

Alignment: **PARTIAL** — the agent layer is ready; the "current state" it infers on is mostly mocked.

- ✓ The agent substrate is genuinely there: 26 MCP tools over SSE (`:8000`) + REST (`:8001`), spanning equipment state, fault history, CMMS, the KG traversal tools above, schematic vision (`kg_extract_schematic` , 3-pass IEC/ANSI), and nameplate extraction. The cascade router is provider-agnostic OpenAI-compatible. This is exactly "agents plugged into digital infrastructure."
- ⚠ But per Walker the agents must infer on **current state** — and current state is the simulated layer (#5). So the agent layer is plugged into infrastructure that's partly powered by `fault-sim` and `simulated=true` rows. The plumbing is real; the live signal is not, outside the bench.
- ☐ The master plan Phase 11 adds the missing unifying tools (`get_asset_context(uns_path)` , `read_tag_value` , `read_tag_history` , `get_fault_windows` , `record_decision_trace_step`) — i.e., the plan knows the agent layer needs a clean current-state read surface and has specced it.

Walker #11 — DTMA → Strategy → Architecture → Minimum Technical Requirements → Current State

Alignment: **PARTIAL** — DTMA and Strategy exist; Architecture is strong; "Minimum Technical Requirements" and the current-state delivery are the soft spots.

- DTMA: ✓ designed / ☐ operationally stub. `docs/specs/dt-scorecard-spec.md` + `mira-web/public/assess.html` (`factorylm.com/assess`) is a 6-dimension, 1–5, CESMII-Smart-Manufacturing-based maturity scorecard with maturity tiers

(Foundational/Developing/Practicing/Leading) and a radar chart. This is a real DTMA-equivalent. But v1 has **no email capture and no backend persistence** — it's a lead magnet wireframe, not a measured assessment instrument.

- **Strategy:** ✅ `STRATEGY.md` (ICP, competitive table, Walker rule) + `NORTH_STAR.md` (flywheel).
- **Architecture:** ✅ `docs/THEORY_OF_OPERATIONS.md`, the master plan, ADR-0012/0013/0017/0018/0021, the UNS+KG specs.
- **Minimum Technical Requirements:** ⚠️ scattered. `docs/mira-ignition-secure-architecture.md` (ADR-0021) is the closest thing to a customer minimum-tech-spec (Ignition module, outbound HTTPS only, tag allowlist). There's no single "here's the minimum your plant needs" customer-facing artifact derived from the DTMA score.
- **Current State:** the journey's destination per Walker — and MIRA's weakest delivery (#5).

2. Section A — What MIRA already has (built & working)

Crediting the real strength, with evidence:

- **UNS implementation** ✅ — `mira-crawler/ingest/uns.py` (path builders), `mira-bots/shared/uns_resolver.py` (resolution, `UNSContext`), `kg_entities.uns_path` ltree. Walker-compliant 2D hierarchy (ADR-0018 sibling model). Resolver works offline (alias tables) with NeonDB enrichment additive.
- **Knowledge Graph** ✅ schema + traversal — `kg_entities` / `kg_relationships` (Hub 001/024/028/029), `relationship_proposals` + `relationship_evidence` (Hub 018), `ai_suggestions` (Hub 027, 6 types). Traversal tools `kg_root_cause_chain` / `kg_impact_analysis` / `kg_traverse_chain` / `kg_maintenance_context` in `mira-mcp/server.py`.
- **Diagnostic engine (Analyze)** ✅ — `mira-bots/shared/engine.py` `Supervisor`, FSM (`fsm.py`), groundedness 1–5 self-critique, citation compliance (observational), cascade inference (`inference/router.py`), BM25+pgvector RAG.
- **Connect/ingest of live data (bench)** 🟦 — `mira-relay/relay_server.py` (HMAC ingest, `saas.yml`), `ignition/webdev/FactoryLM/` (WebDev module + gateway tag-stream), `mira-pipeline/ignition_chat.py` (direct-connection cloud endpoint).
- **Store** ✅ — `knowledge_entries` 25k+ rows, full Hub schema (031 migrations), engine KG migrations (008).
- **Visualize** ✅ — Hub Command Center (UNS tree + live iframe), `/namespace`, `/proposals`, `/discovery`; Ignition Perspective `ConvSimpleLive`; Node-RED conveyor HMI (bench).
- **Pattern detection** ⚠️ — recurring-fault (`detection/recurring_fault.py`), Fault Detective 8-rule engine (`mira-fault-detective/rules.py`), `kg_flag_pm_mismatches`.
- **Agent tool layer** ✅ — 26 MCP tools (`mira-mcp/server.py`), SSE+REST, schematic vision, nameplate OCR, CMMS write-back (Atlas/MaintainX/Limble/Fiix via `cmms/factory.py`).
- **Fieldbus discovery** 🟦 — `plc/discover.py` (read-only TCP+CIP+Modbus FC1-4 scan; RS-485 gated by `--serial-bus-idle`; PR #1586). The "what's on this network" capability Walker's current-state step needs.
- **Commercial scaffolding** ✅ — `/assess` DTMA scorecard, `/pricing` (3-tier), Stripe checkout (`mira-web`), Atlas CMMS auto-provisioning (`saas-activation`).

3. Section B — Designed but not built (the master plan + specs)

The master plan (`docs/plans/2026-06-01-mira-master-architecture-plan.md` , `Status: DRAFT`) is the authoritative gap list. The plan-critical *new* layer — the trace/event/flaky-signal/allowlist tables and their workers — does **not exist**:

Capability	Master-plan phase	Status
<code>decision_traces</code> table + <code>DecisionTraceWriter</code>	Phase 1 schema (<code>032</code>), Phase 8 writer	☐
<code>tag_events</code> append-only diff stream + <code>diff_logger</code>	Phase 1 (<code>033</code>), Phase 5 worker	☐
<code>flaky_input_signals</code> + <code>FlakyInputDetector</code>	Phase 1 (<code>034</code>), Phase 9 worker	☐
<code>approved_tags</code> table (file→table cutover)	Phase 1 (<code>035</code>), Phase 4 enforcement	☐ (today <code>approved_tags.json</code>)
<code>state["uns_context"]["source"]="direct_connection"</code> wired on Ignition chat	Phase 6	☐ (rule documented, code doesn't set it)
Citation enforcement (not just observation)	Phase 7	☐ (<code>citation_compliance.py</code> logs only)
Proposal-transition helpers (<code>proposal_transition.{ts,py}</code> , ADR-0017)	Phase 3	☐
<code>get_asset_context</code> / <code>read_tag_value</code> / <code>read_tag_history</code> MCP tools	Phase 11	☐
First-class ingest API (<code>POST /api/v1/ingest</code>) + MiraDrop v2 (ADR-0019)	Phase 2	☐
Mock tag collector + YAML scenarios (unblocks 5/9/12 w/o hardware)	Phase 4	☐
5-minute flywheel demo + <code>regime8</code> / <code>regime9</code> tests	Phase 12	☐

Specs that are approved-but-unbuilt or partial: `uns-kg-unification-spec.md` (APPROVED 2026-05-07, Phases 1–3 in flight — unifies the three disconnected stores so every chunk gets `equipment_entity_id` and every entity gets `uns_path`); `knowledge-graph-multi-hop-spec.md` (APPROVED, building); `enforcement-layer-spec.md` (Phase 1 warn-only); MiraDrop ingest v2 (ADR-0019, locked, no code).

Known defects that block the journey (`docs/known-issues.md` + memory): Gemini key 403 (cascade falls through); upload→retrieval gap (uploaded manuals not citable); `demo_plc_poller.py` `live_signal_cache` DDL collides with Hub migration 020; recall embedder ops (Bravo Tailscale / VPS localhost) unstable.

4. Section C — Completely missing (blind spots)

These are absent from both code *and* plan, mapped to Walker's framework:

- Live fleet current-state** (Walker #5, the load-bearing one). MIRA cannot answer "which assets are running right now?" across a plant from live data. Bench-only, one cell, via bench tooling. *Partially* addressed by master-plan Phase 4/5, but those are mock-collector-first — they unblock development, they don't yet stream a real plant.
- Predictive layer** (Walker #7 "find patterns → predict future"). No historian, no SPC/Z-score/threshold-exceedance on live tags, no ML anomaly detection, no trend/regression on MTTR/MTBF. Reports KPIs are hardcoded. Walker's "historize → pattern → predict" pipeline is the clearest missing arc. *Note*: MIRA's competitive claim is **grounded diagnostics, not prediction** — so this may be a deliberate non-goal. But against Walker's framework it's a stated absence, and it's where the funded competitors (Augury, TRACTIAN) actually play.
- Digital supply chain** (Walker #3b). No supplier/parts/procurement integration reaching outside the plant. Zero code, zero plan.
- Sell-your-data mechanism** (Walker #3c). "Knowledge Cooperative" is a flywheel slide. No anonymization pipeline, no data-product packaging, no marketplace.
- Sparkplug B decode** (Walker #6/#7 connect). The industry-standard MQTT payload for UNS is a schema *column* (`tag_entities.sparkplug_topic`), not a parser. The UNS-architecture skill and rules reference it; nothing decodes `spBv1.0/...` birth/death/data.
- ERP / MES** (Walker #6 stack). Out of scope by decision — name it so the footprint isn't oversold.
- DTMA → Minimum-Technical-Requirements bridge** (Walker #11). The scorecard scores maturity but doesn't emit a per-customer "here's the minimum your plant needs to start" spec. The link from assessment to architecture is human, not productized.

5. Section D — The garage conveyor as DT proof

The bench = Micro820 + GS10 VFD + photo-eyes + Ignition gateway + MIRA. Honest scoring of what it proves:

DT capability	Demonstrated?	Real / Sim
(a) Current-state capture	YES on the bench	✓ REAL — <code>live-plc-bridge</code> streams real GS10 freq/current/DC-bus/motor from the Micro820 @ 500 ms; Ignition <code>ConvSimpleLive</code> CiP tags update live. ■ bench-only path.
(b) UNS	PARTIAL	⚠ Hub tree shows real <code>enterprise.* kg_entities</code> ; but live MQTT uses a flat <code>demo/cell1/conveyor/cv101/</code> namespace that doesn't reconcile to the ISA-95 tree.
(c) Knowledge graph	PARTIAL	⚠ GS10/Micro820 component templates seeded into <code>knowledge_entries</code> /KG (<code>tools/seeds/gs10-vfd-knowledge.sql</code>); but Fault Detective doesn't write <code>kg_relationships</code> / <code>ai_suggestions</code> from live faults. No self-growing graph.
(d) Pattern detection	PARTIAL	■ 8 real rules (incl. <code>rule_pe101_chatter</code> flaky-wire @ ≥5 dropouts/window with stable peers, conf 0.85) — but PE-101/102/PX-101 sensor faults are REST-injected by <code>mira-fault-sim/sim.py</code> , not wired hardware. VFD/motor state <i>is</i> real.
(e) Agent-assisted troubleshooting	PARTIAL	⚠ Ignition "Ask MIRA" panel exists; <code>mira-pipeline</code> direct-connection path implemented; GS10/Micro820 knowledge seeded. But the end-to-end flywheel demo (<code>docs/demos/...flywheel-demo.md</code>) is unbuilt (Phase 12).

The real fault path is genuinely impressive and honest: `plc/Micro820_v4.1.9_Program.st` has a real 5-second `vfd_err_timer` — two-master RS-485 contention CRC-faults the Micro820's GS10 polls → 5 s watchdog → `fault_alarm` / `error_code=9` / `conv_state=FAULT` → motor stop. The GS10 side mirrors it (`P09.03=5` → CE10 comm-timeout). `MbSrvConf_v4.xml` is the deployed ground-truth Modbus map. The promo videos (`marketing/comic-pipeline/scripts/phase5-video-13-ask-conveyor.yaml`) are *honest* — "This isn't a sandbox. It's a real Micro 820, a real GS10 drive, real photo-eyes."

Gap to a "that's digital transformation" demo (Walker's bar): 1. **Wire the sensors.** Move PE-101/102 faults from `fault-sim` REST injection to real Modbus-mapped DI on the bench, so flaky-wire detection runs on *physical* intermittency. This is the single highest-credibility upgrade. 2. **Reconcile live data to the ISA-95 UNS.** Map the `demo/cell1/...` MQTT topics to `enterprise.garage.demo_cell.*` so the Command Center's live dot means "this real asset, at this UNS path, is running." 3. **Close the self-growing-graph loop.** A confirmed bench fault should auto-propose a `kg_relationship` (e.g., "PE-B16 → TB2-15 loose") into `relationship_proposals` → `/proposals` → verify. That's the "3D graph grows from current state" moment that makes Walker nod. 4. **Build Phase 12.** The 9-step flywheel script (photo → component template → live fault → Slack-confirm → wiring proposal → KG enrichment → next plant inherits 80%) is fully written in the master plan §D7; it's a *recording* task, not a design task — once Phases 4/5/6/9 land.

The demo that earns Walker's "that's DT": Stand at the bench. Open the Command Center — the UNS tree shows `enterprise.garage.demo_cell.conveyor.gs10` with a live green dot driven by *real tag freshness*. Wiggle the PE-101 wire — within seconds a `flaky_signal_alert` appears in `/proposals` , sourced from real `tag_events` . Ask MIRA in the Perspective panel "why is the conveyor confused?" — no asset-confirmation question (direct-connection UNS cert), grounded answer citing the GS10 manual page + the live flicker count + the 5 s comm-timeout. Approve the "loose wire at TB2-15" relationship. Onboard a second (mock) plant with a GS10 — it inherits the component template at 80% structure. That sequence *is* connect → current-state → pattern → root-cause-across-unrelated-data → sell-the-reusable-data. Every box in that chain except the live-data-to-UNS reconciliation and the auto-proposal already exists in some form.

6. Section E — MIRA as a DT enablement platform for customers

Mike sells DT services. Does MIRA move a customer Industry 3 → Industry 4, and can it make 12-of-12 succeed?

The offer stack (exact, from [NORTH_STAR.md](#) / [STRATEGY.md](#) / [mira-web/public/pricing.html](#))

Offer	Price	Delivers
Assessment	\$500 one-time	Floor walk (1 day), 6-dimension AI-readiness scorecard, written gap report + namespace blueprint
Pilot	\$2K-\$5K/mo, 3-mo min	Structure one line/cell: nameplates, manuals, PLC tags, PMs, fault history; MIRA deployed on that scope
Operating Layer	\$499/mo per plant	MIRA in production: unlimited grounded queries, CMMS write-back, continuous structuring, quarterly audit
Enterprise	Custom	5+ plants, on-prem, SLA

Mapping offers to Walker's journey (Assessment → Connect → Current State → Historize → Predict)

DT stage	Offer	Built?	Gap
Assessment (DTMA)	\$500 + /assess	⚠️ scorecard wireframe; \$500 sold via mailto/LinkedIn	No email capture / backend; no self-serve booking
Connect	Pilot	⚠️ Ignition path exists; PLC tag↔asset reconciliation "NOT BUILT" (NORTH_STAR.md)	The Connect deliverable the Pilot <i>promises</i> is the weakest-built layer
Current State	Pilot	⚠️ nameplate/manual structuring ✅ (83k OEM chunks, BM25); PM-schedule extraction "NOT BUILT"	Live current-state is simulated (#5)
Historize	Pilot + Operating	⚠️ CMMS write-back ✅; RCA schema TBD; tribal-knowledge capture "NOT BUILT"	No real historian
Predict	Operating (\$499)	⚠️ grounded answers ✅; no prediction	Differentiation is grounding, not prediction

The pricing contradiction (state it plainly — Mike wants no smoothing)

There are **two live, inconsistent motions**: - **Services-led** on the public site: [pricing.html](#) shows \$500 / \$2-5K / \$499 (assessment→pilot→operating). - **Product-led** in the code: [mira-web/src/lib/stripe.ts](#) wires Stripe to a **\$97/mo beta** subscription; ADR-0014-B ([Accepted 2026-05-20](#)) declares MIRA "product-led, self-serve" and names \$97/\$297 tiers; the ADR itself flags "*Pricing pages are mutually inconsistent* (\$20/\$499 vs \$97/\$297)."

This isn't a rounding error — it's an unresolved strategic fork (services-DT-firm vs self-serve-SaaS) that the codebase and the marketing site disagree on. For a DT-enablement positioning, the services-led stack (\$500 assess → pilot → operating) is the one that matches Walker's journey; the \$97 self-serve motion matches a horizontal SaaS. They imply different products. Pick one as primary before scaling either.

Does MIRA make 12-of-12 succeed instead of 1-of-12?

Today: it de-risks the front of the journey (assess + structure), not the middle (live current state) or the end (predict). The reason 11 of 12 fail, in Walker's telling, is they can't get past current-state capture into sustained data-driven operation. MIRA's honest current position: - ✅ It uniquely nails **document/knowledge structuring** — the 83k pre-indexed OEM chunks (Rockwell/ABB/Siemens/Schneider/Yaskawa) are a real moat, and grounded answers beat a consultant's binder. - ⚠️ It does **not yet sustain live current state** for a customer fleet — the exact wall the 11 hit. The Pilot *sells* PLC-tag reconciliation that is "NOT BUILT." - ❌ It does **not predict** — so it doesn't carry a customer to Walker's data-driven-decisions endpoint.

What MIRA needs to become 12-of-12 (grounded in the master plan, not invented): 1. **Ship Phase 4 + Phase 5** — real tag collector (Ignition-side, customer-deployable) + [tag_events](#) stream. This is the literal "current state" Walker demands and the literal wall the 11 fail at. *Highest leverage.* 2. **Ship Phase 9** — flaky-signal detector — the first true "find patterns on live data" capability, and the demo that proves MIRA crosses Industry 3→4. 3. **Close the upload→retrieval gap + Phase 2/3** — so structuring (the thing customers pay the Pilot for) actually makes manuals citable and grows the KG through the proposal loop. Without this the Pilot deliverable is leaky. 4. **Resolve the pricing fork** — services-led journey vs \$97 self-serve. DT-enablement positioning wants the former. 5. **Make the DTMA→MTR bridge productized** — the [/assess](#) score should emit the customer's "minimum technical requirements + which Walker stage you're at + what the Pilot will connect first." That turns the scorecard from a lead magnet into the on-ramp Walker's #11 describes.

A defensible honest pitch to a Walker-aware buyer: "*We are the maintenance-side UNS + knowledge-graph layer. We structure your messy maintenance reality into ISA-95 + a 3D knowledge graph, ground every answer in your real manuals and (soon) your real live tags, and grow that graph as your techs confirm relationships. We are not your OEE/MES consultant and not a predictive-sensor vendor — we own the maintenance namespace and the agent that reasons over it.*" That claim is **true today for structuring + grounding**; it becomes true for **live current state** when Phase 4/5 ship.

7. Verdict

- **Architecture alignment with Walker: A-.** UNS (2D), KG (3D), agent tools, the explicit ADR-0012 adoption and "Walker rule" — genuinely, deliberately aligned. The KG traversal tools are a near-literal implementation of Walker's 2026 root-cause-across-unrelated-data thesis.
- **Journey alignment with Walker: C.** Built back-to-front. The brain is real; the live nervous system (current state) is bench-only or simulated; prediction is absent; digital supply chain and sell-your-data are doctrine, not code.
- **The fix is already written down.** MIRA's own master-plan critical path starts with current-state capture (Phase 4/5), exactly where Walker says the journey begins and exactly where the 11-of-12 fail. The gap between MIRA-today and Walker-aligned-MIRA is the gap between the master plan's **DRAFT** header and shipped Phases 1/4/5/9.

The strongest move is not new architecture — it's **building the left half of Walker's pipeline (Connect→Collect→current-state) for real, on the bench first**, so the demo, the Pilot deliverable, and the "we get you past the wall the others die at" pitch all become true at once.

Appendix — Primary evidence index

- Master plan: `docs/plans/2026-06-01-mira-master-architecture-plan.md` (§1 baseline, §2 phases 0–13, §D7 demo, §D8 open questions)
- Doctrine: `docs/THEORY_OF_OPERATIONS.md`, `NORTH_STAR.md`, `STRATEGY.md`
- ADRs: `0012` (Walker UNS framework — Accepted), `0013` (schema lineage), `0014-B` (product-led), `0017` (proposal FSM), `0018` (component siblings), `0021` (Ignition-module-first)
- UNS: `mira-crawler/ingest/uns.py`, `mira-bots/shared/uns_resolver.py`, `.claude/rules/uns-compliance.md`
- KG + agents: `mira-mcp/server.py` (26 tools), Hub migrations `018 / 025 / 027 / 029`
- Connect/store: `mira-relay/relay_server.py`, `ignition/webdev/FactoryLM/`, `mira-pipeline/ignition_chat.py`, `docs/migrations/001`, Hub migrations `019 / 020 / 025 / 030 / 031`
- Engine: `mira-bots/shared/engine.py`, `fsm.py`, `inference/router.py`, `rag_worker.py`, `citation_compliance.py`
- Patterns: `mira-bots/shared/detection/recurring_fault.py`, `mira-fault-detective/rules.py`, `mira-fault-sim/sim.py`
- Bench: `plc/Micro820_v4.1.9_Program.st`, `plc/MbSrvConf_v4.xml`, `plc/live-plc-bridge/bridge.py`, `docker-compose.fault-detective.yml`
- Commercial: `mira-web/public/pricing.html`, `mira-web/public/assess.html`, `docs/specs/dt-scorecard-spec.md`, `mira-web/src/lib/stripe.ts`, `docs/adr/0014-product-led-wedge.md`